Group Project

ME 326: Collaborative Robotics
Stanford University
Winter 2026

Competition and Deadline: March 12, 2026
Final Team Report and Website Due: 11:59p on March 14, 2026
Worth: 40pts

Contents

1 Project Goal and Rules

1.1 Problem: Collaborative Robotic Assistant

The goal of the project will be to make a mobile robot an effective collaborator and assistant for a human counterpart. Our class will use the TidyBot++ base with a custom bimanual manipulation

setup using the 6-DoF WidowX Arms. The system will also have a head-mounted Intel RealSense depth camera and a LiDAR for localization.

In this project, you will use natural communication methods (e.g. natural language, or gestures) to communicate an intended task to the robot. You will use robot perception (vision) to perceive the environment, audio to hear from the person, and use existing libraries (or those you may develop) to take this input and determine the task the human is requesting, then using computer vision, perceive the environment, then control the motion of the robot to complete the requested task.

The three tasks that every team must complete are

1. **Object Retrieval:** Teams will use language to request the object to bring a desired object back to a particular location (e.g. "locate the apple in the scene and retrieve it"). The robot must interpret this verbal command, scan and navigate the scene, locate the apple in a cluttered environment, navigate to the apple, then pick it with the manipulator, and return to the starting position

2. **Sequential Task:** In this task, teams may use natural language to request a series of actions be performed in the environment. For instance, for cleaning, the user may ask that the object locate an object (e.g. a red apple) and place it in a brown basket located in the scene: "find the red apple and place it in the basket."

3. **Group Chosen Task:** In this task, the team will be given the freedom to pick a collaborative task (the team must propose and have it approved by the teaching team by the end of week 4). The task should involve perception, planning, and control (navigation and manipulation), and have some real-world potential value.

2 Deliverables

2.1 Demonstration Day(s)

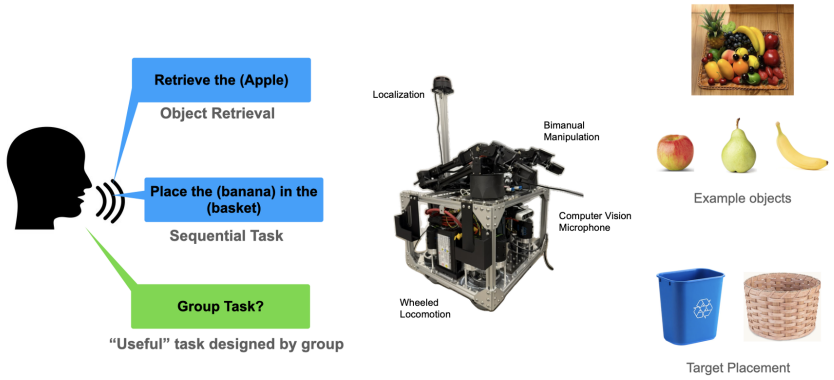
During week 10, there will be a demonstration day, where teams will present their robots' ability to perform each of the three above tasks through a presentation with recorded instances.

2.2 Group Project Report

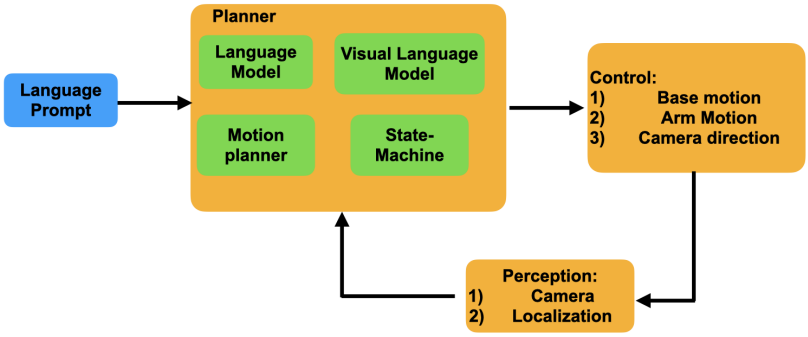
The team must prepare and submit a group report through a portfolio website that includes:

- Title page with group name and members
- Contributions of each teammate
- Project description:
  - Problem Statement
  - Method used by the team to complete the tasks (state machines, code architecture/flow)
  - Videos/images of working system (simulation and real)
  - Accessible code base (GitHub)

It is suggested that teams use Github pages (tie to external email as well for longevity), or Google Sites (if tied to school email may eventually not be supported), or Weebly be used for ease of development. Examples from past years can be found at the bottom of the page here: `https://arm.stanford.edu/courses/collaborative-robotics/`



(a) Overview of task types



(b) Overview of system diagram

Figure 1: **Project Description.** Teams will use natural language and optionally human gestures to communicate a task objective to the robot. The tasks will include a) an object retrieval task (one of N objects) b) a sequential task (e.g. placing an object in a container) and c) a team-defined task (to be proposed by the team and approved by the teaching team).

2.3 Individual (Team Contribution) Report

On canvas, each team member will be required to submit a 1-page document outlining:

1. What responsibilities in the team did you lead and take ownership?
2. For each of your team members, what responsibilities did they lead and take ownership of?

3 Project Logistics

3.1 Getting Started: Lab location

We are using the TidyBot++ from Princeton University, Stanford University, and Dexterity `https://tidybot2.github.io/`, `https://tidybot2.github.io/docs/usage/` paired with WidowX Arms from Trossen Robotics `https://docs.trossenrobotics.com/interbotix_xslocobots_docs/index.html`

The d'Arbeloff classroom is located in Building 02-524 (452 Escondido Mall Stanford, CA 94305). **Teams will be allowed to work on the physical robot during the assigned lab hours.**

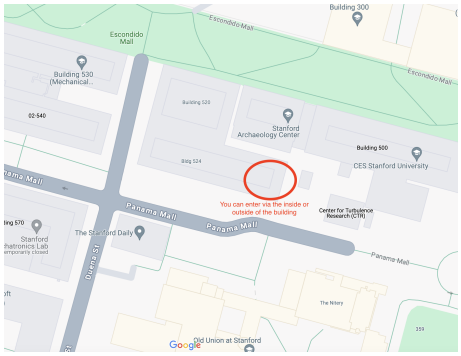


Figure 2: Location of d'Arbeloff Lab, 452 Escondido Mall

3.2 Getting Started: Docker or VM Install for running robots and simulation

You will want to install the necessary code on your laptop(s). **It is recommended to use Docker Desktop to help manage images/containers.**

- Docker Desktop app install for Mac
- Docker Desktop app install for Windows
- Docker Desktop install for Linux
- VMWare Workstation for Windows and Mac
- UTM VM for Mac
- Parallels VM for Mac

Please see the teaching team's accompanying instructions on the wiki Github for setting up Docker and downloading images for this course – the link is HERE

3.3 Direct Install (no Docker)

If downloading and accessing directly, you can find the installation instructions on our class Github: `https://github.com/Stanford-ARM-Lab/collaborative-robotics-2026`

The repository includes a complete ROS2 Humble workspace with MuJoCo simulation. Follow these steps on Ubuntu 22.04:

1. Install ROS2 Humble:

```
# Add ROS2 repository
sudo apt update && sudo apt install -y software-properties-common curl
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
    -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) \
    signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
```

```
    http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && \
    echo $UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list

# Install ROS2 Humble desktop
sudo apt update
sudo apt install -y ros-humble-desktop

# Install additional ROS2 packages
sudo apt install -y ros-humble-xacro ros-humble-robot-state-publisher \
    ros-humble-joint-state-publisher ros-humble-rviz2
```

2. Install uv (Python package manager):

```
curl -LsSf https://astral.sh/uv/install.sh | sh
source ~/.bashrc  # or restart terminal
```

3. Clone and build the repository:

```
git clone https://github.com/Stanford-ARM-Lab/collaborative-robotics-2026.git
cd collaborative-robotics-2026

# Install Python dependencies (MuJoCo, numpy, opencv, etc.)
uv sync

# Build ROS2 workspace
cd ros2_ws
source /opt/ros/humble/setup.bash
colcon build
```

4. Source the environment:

The repository provides a `setup_env.bash` script that handles all environment setup (ROS2, uv packages, and workspace). Use this each time you open a new terminal:

```
cd ~/collaborative-robotics-2026/ros2_ws
source setup_env.bash
```

Alternatively, add this alias to your `~/.bashrc` file for convenience:

```
alias tidybot='cd ~/collaborative-robotics-2026/ros2_ws && source setup_env.bash'
```

5. Test the simulation:

```
ros2 launch tidybot_bringup sim.launch.py
```

3.4 Physical Robot Access:

The Bimanual TidyBot++ robots are configured to use ROS2's native DDS networking with Cyclone DDS. Follow these steps to connect:

- Join the lab WiFi network (ask teaching team for credentials)
- Install Cyclone DDS on your machine (one time):

```
sudo apt install ros-humble-rmw-cyclonedds-cpp
```

- Add the following to your `~/.bashrc` file:

```
export ROS_DOMAIN_ID=42
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
```

- To ssh into the robot (password: *locobot*):

```
ssh -X locobot@<ROBOT_IP>
```

Note: the `-X` option enables X11 forwarding to display visuals on your screen. To simplify SSH access:

1. ssh-copy-id: Generate an SSH key for passwordless login.
2. tmux (installation): Persist terminal sessions across network interruptions (cheatsheet).

- **On the robot**, start the hardware drivers:

```
export ROS_DOMAIN_ID=42
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
cd ~/collaborative-robotics-2026/ros2_ws
source setup_env.bash
ros2 launch tidybot_bringup robot.launch.py
```

- **On your machine**, verify connection:

```
source ~/.bashrc  # Load ROS_DOMAIN_ID and RMW settings
cd ~/collaborative-robotics-2026/ros2_ws
source setup_env.bash
ros2 topic list   # Should show /cmd_vel, /joint_states, etc.
```

3.5 Remote Access via Tailscale VPN (Optional)

Tailscale enables secure remote access to the robot from anywhere - your home, coffee shops, or different networks. This is the easiest way to control the robot remotely without port forwarding or complex VPN configuration.

3.5.1 Why Use Tailscale?

- **Access from anywhere:** Control the robot from home, coffee shops, or different networks
- **No port forwarding needed:** Works through NAT and firewalls automatically
- **Encrypted connection:** All traffic is secured via WireGuard
- **Simple setup:** Just install and authenticate on both machines
- **Direct peer-to-peer:** Low latency compared to traditional VPNs
- **Works with ROS2:** DDS multicast works over Tailscale network

3.5.2 Installation and Setup

Step 1: Install Tailscale on Your Machine

Choose the appropriate installation method for your operating system:

For Linux:

```
# Install Tailscale
curl -fsSL https://tailscale.com/install.sh | sh

# Start and authenticate
sudo tailscale up
```

For macOS:

- Download from `https://tailscale.com/download/mac`
- Or install via Homebrew (CLI-only):

```
brew install tailscale
sudo brew services start tailscale
sudo tailscale up
```

For Windows (if using WSL2):

- Install Tailscale on Windows (not inside WSL)
- Download from `https://tailscale.com/download/windows`
- WSL2 will automatically have access to the Tailscale network

Step 2: Authenticate with Course Account

After running `sudo tailscale up`, a link will appear in your terminal. Follow these steps:

1. **Click on the Tailscale authentication link** that appears in your terminal
2. **Login with GitHub** using the course credentials:
   - GitHub Email: `collab-robotics-stanford@protonmail.com`
   - GitHub Password: `CollabRobotics2026$`
3. After GitHub login, you will be prompted for a **verification code**
4. **Access ProtonMail** to retrieve the verification code:
   - Go to ProtonMail (`https://mail.proton.me`)
   - Login credentials:
     - Email: `collab-robotics-stanford@protonmail.com`
     - Password: `CollabRobotics2026$`
   - Check inbox for GitHub verification email
   - Copy the verification code
5. **Enter the verification code** in the authentication prompt

6. **Wait for approval:** Your device will need to be added to the course Tailscale network by an administrator. You will receive a notification once approved.

Step 3: Set Up VSCode Remote SSH (Recommended)

Using VSCode's Remote SSH extension allows you to develop on the robot as if it were local, with full IDE features.

1. **Install the Remote - SSH extension in VSCode:**
   - Open VSCode
   - Click on the Extensions icon in the left sidebar (or press `Ctrl+Shift+X` / `Cmd+Shift+X`)
   - Search for "Remote - SSH"
   - Install the extension by Microsoft
2. **Add the robot as an SSH host:**
   - Press `Cmd+Shift+P` (macOS) or `Ctrl+Shift+P` (Windows/Linux) to open the Command Palette
   - Type "Remote-SSH: Add New SSH Host" and select it
   - Enter the SSH connection command:

     `ssh locobot@100.106.67.118`

     Replace `<ROBOT_TAILSCALE_IP>` with the robot's actual Tailscale IP address (provided by teaching team)
   - Select the SSH config file to update (usually `~/.ssh/config`)
3. **Connect to the robot:**
   - Press `Cmd+Shift+P` / `Ctrl+Shift+P` again
   - Type "Remote-SSH: Connect to Host" and select it
   - Choose the robot from the list (should show `locobot@<ROBOT_TAILSCALE_IP>`)
   - Enter password: `locobot`
   - A new VSCode window will open connected to the robot
4. **Optional - Set up SSH key for passwordless login:**

```
# On your local machine, generate SSH key (if you don't have one)
ssh-keygen -t ed25519 -C "your_email@example.com"

# Copy your public key to the robot
ssh-copy-id locobot@100.106.67.118

# Now you can connect without entering password each time
```

Step 4: Verify Connection and Test ROS2

Once connected via Tailscale (and optionally VSCode Remote SSH):

```
# Check Tailscale status
tailscale status

# SSH to robot (if not using VSCode Remote SSH)
ssh locobot@100.106.67.118

# Configure ROS2 environment
export ROS_DOMAIN_ID=42
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
cd ~/collaborative-robotics-2026/ros2_ws
source setup_env.bash

# Test ROS2 connection
ros2 topic list
```

3.5.3 Important Notes for Remote Access

- **Enable image compression:** When controlling the robot over Tailscale, always enable image compression to reduce bandwidth usage:

  ```
  ros2 launch tidybot_bringup real.launch.py use_compression:=true
  ```

  This reduces camera bandwidth from ∼14 MB/s to ∼1-2 MB/s.

- **Connection type:** Use `tailscale status` to check if you have a "direct" (peer-to-peer) or "relay" connection. Direct connections provide better latency.

- **Shared credentials:** The course GitHub and ProtonMail credentials are shared among all students. Do not change these passwords.

- **Device approval:** Contact the teaching team if your device is not approved within 24 hours.

- **VSCode Remote SSH benefits:**
  - Edit files directly on the robot with full IDE features
  - Integrated terminal runs commands on the robot
  - Git integration works seamlessly
  - Extensions (Python, C++, ROS) work on remote files

3.5.4 Troubleshooting Tailscale Connection

If you cannot see ROS2 topics after connecting via Tailscale:

1. **Verify both machines are connected:**

   ```
   tailscale status
   # Both robot and your machine should show as "active"
   ```

2. **Test network connectivity:**

   ```
   ping 100.106.67.118
   ```

3. **Ensure ROS environment variables match:**

```
echo $ROS_DOMAIN_ID  # Should be 42
echo $RMW_IMPLEMENTATION  # Should be rmw_cyclonedds_cpp
```

4. **If multicast doesn't work,** create a Cyclone DDS config file pointing to the Tailscale interface (see course wiki for details).

5. **VSCode Remote SSH issues:**

   - If connection hangs, try restarting VSCode
   - Check that Tailscale is running: `tailscale status`
   - Verify SSH works from terminal first: `ssh locobot@100.106.67.118`
   - Check VSCode's SSH logs: View → Output → Remote-SSH

- The repository includes the following ROS2 packages:
  - **tidybot_bringup**: Launch files and test scripts for simulation and real hardware
  - **tidybot_control**: Arm, base, and pan-tilt controllers
  - **tidybot_description**: URDF/XACRO robot model
  - **tidybot_ik**: Motion planning and inverse kinematics (using mink library)
  - **tidybot_msgs**: Custom ROS2 message types (ArmCommand, GripperCommand, etc.)
  - **tidybot_mujoco_bridge**: MuJoCo simulation bridge for ROS2
  - **tidybot_client**: Remote client utilities and DDS configurations
- See docs/remote_setup.md for detailed network configuration and troubleshooting.

3.6 Project Teams

Project teams have been assigned and you can find your team in the Canvas project page, based on your desired selected group mates. Everyone in the team is expected to contribute to robot coding and algorithmic planning for the group project, the final report must outline the contributions of all team members. Additionally, teams will prepare websites that showcase their work. Websites and the report should have the following components: 1. Problem Statement 2. Approach Overview 3. Code/System architecture (details of your method) 4. Results (video, plots) 5. Team biography. (see previous year examples at the bottom of this page: `https://arm.stanford.edu/courses/collaborative-robotics/`)

3.7 Bimanual TidyBot++

As retrieved from their website: `https://tidybot2.github.io/`

> The Bimanual TidyBot++ is an open-source holonomic mobile manipulation platform designed to support research in robot learning, navigation, and real-world manipulation. The system enables researchers, educators, and students to focus on high-level algorithm development rather than low-level hardware integration. Leveraging open-source software and modular design, TidyBot++ supports rapid experimentation with mobile manipulation and data collection pipelines.

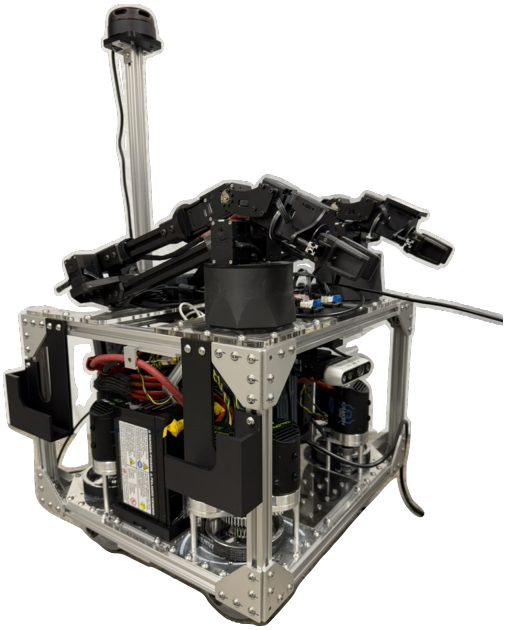


Figure 3: Modified TidyBot++ from Princeton University, Stanford University, and Dexterity: Consists of a TidyBot++ base, it has an onboard Intel Nuc computer, an Intel Realsense depth camera, two WidowX 6DOF arms, and RPLIDAR laser range scanner.

> Our platform is based on TidyBot++, but includes several key modifications. The robot is configured with two WidowX-250 robotic arms, repurposed from the LoCoBot platform, enabling bimanual manipulation capabilities. For perception and navigation, the system is equipped with a LiDAR sensor and an Intel RealSense depth camera, supporting mapping, obstacle avoidance, and manipulation-centric perception. The holonomic base allows independent control of planar motion (x, y, θ), facilitating precise navigation and positioning during manipulation tasks.

3.8 ME326 Computing Requirements

Members of your team should have the ability to locally run Linux and ROS on their computer to serve as the *Remote* computer for Locobot control. Teams will have optional access to an external GPU which will be a shared resource for the class during lab hours. Outside of lab-hours, teams may leverage their Google Credits for testing learning components of algorithms as applicable.

3.9 Project Timeline and Benchmarks

3.9.1 Week 3-4

Project teams have been formed, and teams log onto the robot to perform basic robot motion following the QuickStart instructions.

1. Teams should set up a Github repository for group collective coding and project integration. (if team members are unfamiliar or need a Github refresher, this is a great tutorial: `https://www.atlassian.com/git/tutorials/learn-git-with-bitbucket-cloud`)

2. Teams should be able to demonstrate mastery of working in simulation to get the robot to move towards an object and pick up an item

3. **Teams should propose their 3rd project to the teaching team and have approval**

3.9.2 Week 5-6

By this week teams should:

- Be able to move the robot base to a specified location
- Use the camera to see the position of the object
- Use mink (tidybot_ik) to plan motion to an object and perform grasping action (without floor collision).

3.9.3 Week 7-8

By this week, teams should have integrated language prompts and visual language model to take command from a human user, and perform localization of an object and retrieval and pick-and-place. Teams may use this week to further refine camera coordinate transforms. Teams should have success demonstrations from each subcategory (moving base to desired location, identifying an object with the camera, and grasping a seen item). Teams should also have their third task under-way.

3.9.4 Week 9

By this week, teams should implement the methods and test fully integrated pipelines from perception to navigation to action.

3.9.5 Week 10

Finishing touches and Demonstration day!!

3.10 Further Information:

Some useful tools are:

3.10.1 Visual Language Models.

There are many options for visual language models, and they are useful for bringing language and context to images which can be used to determine robot action. You are free to use whichever tools you like to accomplish the goal, however the class provides financial support through Google credits (so tools like Gemini may be useful if you want to leverage those credits). It is also possible to leverage VLM tools that perform edge-computing (no need for a server, the model is downloaded locally).

An example of a local-VLM is the use of Yolov11 with Clip as shown in Fig. 4.

- Yolo (v11): `https://docs.ultralytics.com/models/yolo11/#overview`
- Clip: `https://github.com/mlfoundations/open_clip`, torch integration (`https://pypi.org/project/open-clip-torch/`)

Figure 4: Example of using Yolov11 with clip to find "banana" in the image. The center of such a bounding box could then be used to drive robot motion and grasping.

3.10.2 Grasping an Object.

So if the object of interest is located, and the robot navigates to the object, how can a successful grasp be executed? This requires a) having a target grasp pose for your gripper b) being able to move your gripper to that pose.

- **GraspAnything.** This tool proposes candidate grasps around objects of interest (specified through language, and image passed to the model): `https://airvlab.github.io/grasp-anything/docs/grasp-anything-6d/`
- **TidyBot2 Motion Planner (tidybot_ik):** The repository includes a custom motion planning node that uses Mink for inverse kinematics. Unlike MoveIt2, this is a lightweight solution designed specifically for the WX250s arms.
- **Mink documentation:** For advanced trajectory optimization and IK configuration: `https://github.com/kevinzakka/mink`